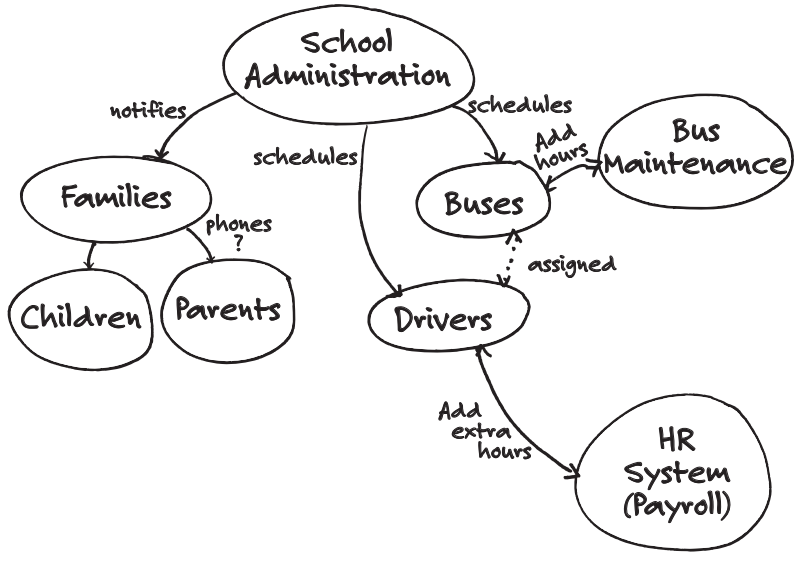


Figure 18-2 Context diagram for a school and bus system

Let's apply the concept of levels of precision to an organization with multiple applications and several teams working to deliver the product enhancement. Each team tests their features to the best of their abilities, but it would be naïve to think that there are no dependencies between the teams or that they can test everything about a story or feature within their own team. In our experience, products within an enterprise have very complex systems, which results in very complex test environments.

A System Test Team and Environment

One of the options when there are multiple teams working on the same product is to create a system test team, perhaps consisting of both programmers and testers. This team works closely with the other delivery teams. Everyone participates in a project inception or high-level planning meeting at the start of the project. Each team has its own continuous integration (CI) environment but also integrates its code into a master build for the entire product. The system test team continually tests the "potentially shippable" product delivered at the end of each iteration, or possibly more frequently. The post-development testing

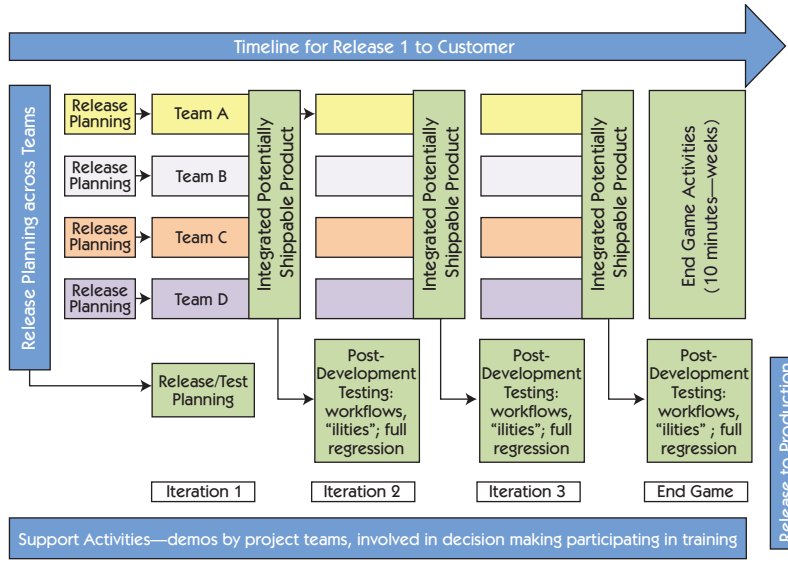


Figure 18-3 One possible solution for multiple teams

and end game activities aren't hardening or bug-fixing iterations. (See *Agile Testing*, pp. 456–57, for more on the end game.) Rather, they are an integral part of the development process, testing the entire system end to end in a production-like environment, which may not be available to each individual team. In Figure 18-3, we show four teams working on separate features, each enhancing the product as a whole.

In our school bus scheduling example, each feature would be assigned to a different team. For example, Team A might take on the feature "Locate buses that are in trouble," while Team B has "Contact driver and assign a replacement bus." Each team would figure out what stories they needed in release planning and estimate approximately how long it would take to complete their feature. However, as you can imagine, there are dependencies between teams, so they also need to consider how to test those. Once the teams understand their features, they come together, think about testing at the higher release level, bring up dependencies, and make decisions about how to test them (see Figure 18-4). Sometimes, test managers are a natural fit in large organizations, helping to coordinate the dependencies between teams.

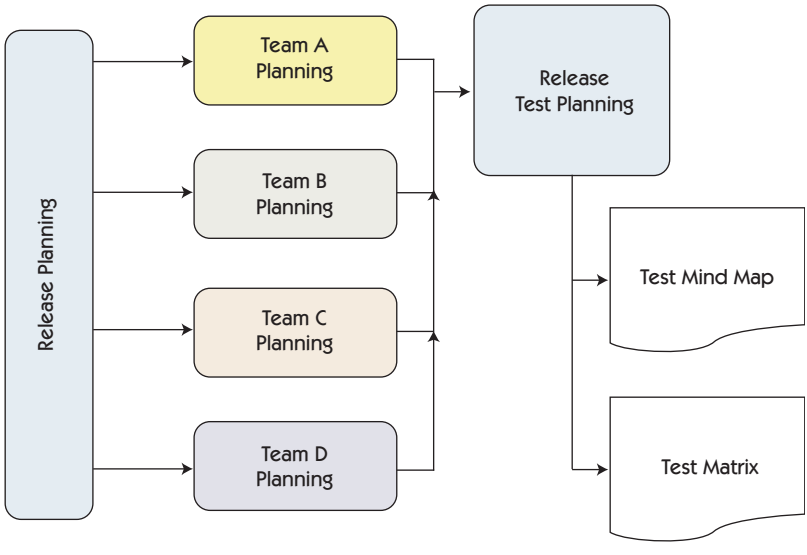
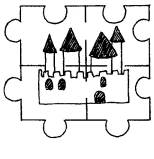


Figure 18-4 Test planning—product level

Testing activities for stories should be completed within the team as much as possible. Sometimes it is hard to complete testing, because a team may not have a complete production-like system test environment—it may be too expensive. Stories are small and granular, so a team needs to be able to get them to Story Done within the iteration.



Since features may consist of many stories, it may not be possible for an individual team to completely test a feature in their own test environment, for example, making sure that the extra hours for the substitute driver can be tested only in the full system environment that has access to the human resources (HR) system. This might be part of Feature Done, and not Story Done.

However, since the delivery team is ultimately responsible for the completion of that feature, there needs to be a way to show when a feature needs more testing and when it is completed, so communication between teams is essential. Validation that the product is shippable can be completed by a system test team that has access to a full, production-equivalent system that allows them to take advantage of economies of scale for types of testing that are difficult to justify in individual feature

teams. For example, performance, load, interoperability, and browser compatibility testing may require specialized test environments. Full automated system regression testing may be part of this system test team's responsibility as well. During the end game, we would suggest that all teams participate in the activities necessary to take the release to Release Done.

There are a couple of different ways to manage a system test team. One way is to have programmers as part of the team to help identify and immediately fix issues that are found. Different organizations may choose to define different processes to deal with defects and issues. Sometimes, any defects found will go back to the originating team. However, this sometimes can be a problem; if there is no clear-cut owner, teams may defer the defect to another team, which can degenerate into a blame game. Lisa worked with a 25-team company where defects caught by the CI were brought up in a "helps-only" Scrum-of-Scrums. A ScrumMaster would speak up and say his or her team would take responsibility. As with any agile process, finding and fixing defects needs to be a short feedback loop.

Let's go back to Geoff Meyer's story of how Dell handled scaling up its agile testing.

Growing Up—Dell, Part 4

In mid-2011, we began to apply agile practices to our first large-scale projects. One project had nine Scrum teams, and the other had 15. We formed feature-based teams, with embedded testers developing in-sprint test automation for user stories (just as we had for our smaller projects).

However, we determined that this level of testing was going to be insufficient for such a large-scale effort, since no single Scrum team had visibility to the usage of the application from a customer perspective. Thus, we introduced the concept of software system test (SST), which took a more traditional black-box approach to testing, focusing on the user interface of completed features. This approach included automation of the UI test cases. Our goal was to start this testing as early as possible—in parallel to the sprints—but not so early as to hinder the development of in-progress user stories (realistically no earlier than the second or third sprint).

Another aspect of these large-scale projects that the test team needed to address was testing new features, in the form of user stories, across multiple generations of existing servers. This is where the in-sprint automation at the service level truly paid off. We empowered Scrum teams that had user stories encompassing a large compatibility matrix to limit their acceptance tests to one or more reference configurations.

We then established an "extended sprint test" team that took accepted user stories (which included automated test scripts) as input. This team then set up the physical test environments and executed the automated test scripts against these extended compatibility configurations to complete the user stories (see Figure 18-5).

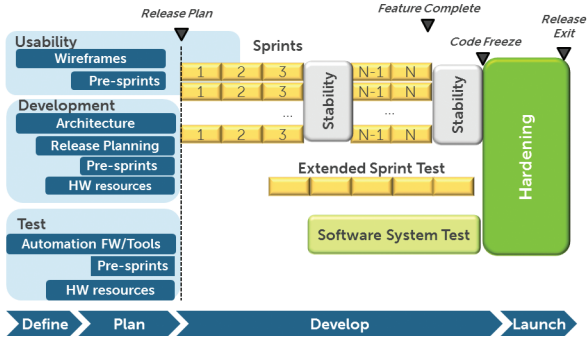


Figure 18-5 Dell's agile adaptations

During the 2012–13 stages of our agile test automation journey, we made our initial foray into Lisa and Janet's fourth quadrant of testing: nonfunctional tests. The emphasis was to apply our automation skills to those areas where manual efforts generally proved impractical. Longevity, scalability, and performance baselining were areas that had received limited coverage in the past, but we were able to build them into the agile development process and run them in parallel to our sprints. We also applied automation to time-consuming test tasks unrelated to test cases.

You can see from Geoff's story that it took time for the large development and test organizations at Dell to discover new ways of working through adaptation and experimentation. The extended sprint test team took advantage of economies of scale and worked in parallel with the Scrum teams to provide feedback as quickly as possible. Often teams

need stabilization iterations while they are adapting, but over time, the need for them should decrease.

Consistent Tooling

There are several other practices that can enhance enterprise success. For example, we can take advantage of the current tools available to organizations. In Chapter 17, "Selecting Test Automation Solutions," we included a section on selecting tools for automation. However, it is common to have existing tools in place, and many organizations hesitate to change. There needs to be a compelling reason to change, so teams need to be able to articulate those benefits.

Coordination through CI

Continuous integration with automated regression testing across the teams with all teams working on the same code base ensures that most integration issues are caught as soon as the code is checked in. Achieving this may take lots of time and effort, but the payback is more than worth the trouble. Lisa worked at a company where it took a few years to get a system-wide CI running just a couple of times a week. But once they achieved this, it was a matter of months to get it running twice a day, and problems were fixed the same day by whichever teams were responsible for the regression failures.

One of the biggest problems that large organizations face is prioritizing bug fixes. Who fixes the defects found by the automated regression tests? How do we determine what the priority is? Many organizations have a production support team that is responsible for fixing customer-reported production defects, but no one cares about the internal defects found. Automated regression test failures must be fixed immediately by the team that checked in the code that caused the failed test; otherwise, the benefit of the fast feedback is lost. Teams need to decide how to handle bugs found during development or in production. As we discussed in Chapter 14, "Technical Debt in Testing," bugs that linger in a defect-tracking system add to the team's technical debt burden.

Use your CI to communicate the results of automated functional regression tests across the development organization. There are several possibilities for managing test failures. One is to have a system test team

investigate failures, decide which delivery team is responsible, and assign the broken test to them. They need a good overview of features being developed so they may have the best insight into what may have broken the tests. Another option is to rotate which team looks at the regression failures. The disadvantage to this approach is that the person or team responsible for checking the results may not know what the other teams are working on and may find it hard to determine the source of the problem. The advantage is that it shares the responsibility of investigating failures and may offer a bit of cross-training. There's no one right approach, so experiment with alternatives.

Version Control Approaches

We like having all teams work on the same code base. This not only mitigates integration risks but also reduces testing risks. We know the automation test suite runs (at a minimum, nightly) on the code so we catch issues early. We do our exploratory testing on the same code base so we know how it behaves on each deploy. Each delivery team has its own test environment, but shared environments may be used as needed. If there is a production defect and we have to do an emergency fix and deploy, we have only two branches to test: production and the main development branch. Figure 18-6 shows a chart of environments Lisa's team tracked while they upgraded their Ruby and Rails versions.

Another approach, which we've seen in some large organizations, is for individual teams to work on separate branches and merge to the main trunk at the end of the iteration. Although there may be extra trouble fixing integration issues for the merge, this approach can work well, as long at the version control system handles merges correctly. One disadvantage to this process is having to decide which branch gets the production fix in emergency situations. Which team will make sure it gets tested before the merge and after the merge? Who keeps track of making sure testing is done in the production branch as well as the development branch? There is a higher risk of something slipping through the cracks when using this strategy, although it can be mitigated if the system test team has a good understanding of the risks and dependencies between teams. The risk is also reduced if the team has dependable test automation coverage and can follow up with some exploratory testing on risk areas.

Environment	Owner	Deployed by	App 1	App 2	Rails	Ruby
DEV	devs	Automated	4.1.0_124	2.1.0	3.2	2.1.1
TEST	testers	Tester pairs	4.1.0_122	2.1.0	3.2	2.1.1
AUTO	DevOps	Automated	4.1.0_124	2.1.0	3.2	2.1.1
SHARED	DevOps	Deploy pair	4.1.0_120	2.1.0	3.2	2.1.1
LOAD	DevOps	Deploy pair	4.1.0_120	2.1.0	3.2	2.1.1
STAGING	DevOps	DevOps	4.1.0_110	2.1.0	2.3	1.87-2012
PRODUCTION	Ops	Ops	4.1.0	2.1.0	2.3	1.87-2012

Figure 18-6 Make your test environments visible.

Test Coverage

One of the risks of multiple teams working on the same product is potential overlaps and gaps in testing. In an ideal world, all features would be self-contained with no dependencies on other features. However, we do not live in an ideal world, especially when it comes to large corporations. They likely have grown over the years through mergers and acquisitions. Coordination of testing between teams that may have originated in different companies, working on separate software products, can become a nightmare.

There are tools now that can help you understand what test coverage your system has. At a recent conference, Janet saw a demonstration of a product designed to help visualize where the gaps in testing might be. Understand your problems and your risks, and then see if you can develop or adopt a tool to help solve it. No tool will address all your test coverage problems.

We encourage teams to experiment, but always keeping in mind that the goal is not to buy more tools, or automate more and more tests, but to reduce risk by fast feedback. This means you want to address integration issues as early as possible and avoid finding them during the end game immediately before release.

MANAGING DEPENDENCIES

Teams try to break up features and stories so there are no dependencies, but that is almost impossible, especially when dealing with enterprise solutions. There are both external and internal third-party vendors, other delivery teams, or even customers that teams may depend on before they can deliver their stories and features.

Working with Third Parties

One of the bigger issues that large organizations have is working with development organizations outside the immediate team. There are both internal and external third parties, and we encourage you to think of them similarly. External vendors may include organizations such as credit card providers over which you have no control, or smaller product companies that are willing to work closely with your team. Internal third parties may include your database team or the accounting department. These internal groups can be as difficult to work with because we expect them to respond when we want them to. However, they have their own sets of priorities and may not respond as expected.

Agile practices for developing in partnership with third-party vendors are becoming well established. Techniques such as moving to an output-and-outcome-focused contract are becoming more frequent (see McDonald, 2013). Perhaps not surprisingly, while practices for integrating third parties into the programming work of a project have emerged, there is little agreement on such practices for testing of agile projects.

Janet's Story

Lately it seems that many of my clients fall into the large enterprise category. Sometimes, one of the first red flags I see is a column in their story board for blocked stories or tasks. It says to me that they are planning to have blocked stories. I would like to see that as an exception rather than a rule.

One piece of advice I give to these teams is to watch out for stories that have a hard dependency on another team or an external third party. I suggest the team take the story out of the iteration until the dependency is removed. That way, the team can focus on other stories instead of waiting for another team to deliver some code or a third party to deliver a promised deliverable.

One way to remove dependencies is to have a "Ready" column or status on your stories. For example, if your story has a dependency where you need to have a database administrator help you with some new database tables, or you need the user experience group to create a wireframe, keep the story in the backlog until you have the necessary artifact or commitment from them to work with you on your story. Contact those groups in advance so they are prepared to provide their deliverables before the story is prioritized. This is part of your story readiness planning. Work to remove these dependencies during planning. Some teams hold meetings specifically to discuss dependencies prior to their iteration planning to deal with situations such as those we described.

Lisa's Story

My last team started doing something similar when we had a lot of stories where the business folks didn't have the business rules ready. They figured we knew the domain so well that we could just make up the rules ourselves. Um, no! We pushed back. If we didn't have all the requirements, mock-ups, and business rules needed for a story by day two of the sprint, that story got taken out, and we used the time to work on our own initiatives to manage technical debt. When business stakeholders realized that not having everything to us by the start of the iteration meant a two-week delay for the story, they were more disciplined about preparing ahead.

When you're working with external third parties, automation can actually work to your advantage. Get the API description from the vendor and create test harnesses to replicate the vendor's inputs to your system, and then you can test your system behavior with automation. This allows you to test your system without actually having the third-party system in place. Yes, there will be changes along the way. However, you will be further ahead and understand what the issues are more easily since you have defined behavior. One of the biggest complaints we hear from teams is, "I can't get that information." Try asking for small pieces at a time if you really need to start developing without having the full API. You might be surprised at how opening the door and starting the conversation creates a better working relationship with the third party, whether it's another team within your corporation or an outside supplier.

Accepting new third-party applications can be scary if you don't trust their changes. Work to improve that trust, but at the same time consider how you can make your team feel more confident about accepting the latest changes. Perhaps create a test environment where you can compare what the system did before the change with how it handles critical functionality after the change. Think about sharing your tests with the vendor to try to get a common understanding of what is being delivered. The vendor may even learn to test before delivering.

Think of all your stakeholders, including your customers, the people actually using your system. If you are an external third party delivering a product, think about how you can make your customers confident in what you deliver.

Involving Customers in Large Organizations

Testing in enterprise systems requires collaborating not only within each development team but also with many outside groups. The most important of these third parties is, of course, our customer. In many corporations, development teams are far removed from their end users. Getting customers involved in the testing and developing process may require extra creativity.

User Acceptance Testing in the Enterprise

Susan Bligh, *a business analyst in Alberta, Canada, tells her story about what happened when the delivery teams didn't include their customers in their testing and how mistakes slipped through the cracks.*

One of the biggest issues we encountered when I was a business analyst working on a major company-wide software implementation was how to integrate the design across all the many functions (silos) of the program. We implemented six different functions that integrated with each other, including Finance and Accounting, Supply Chain, Asset Management, Planning and Budgeting, Capital Projects, and Human Resources.

At the beginning of the program, we were well aware that we needed to have an integrated design that spanned all of the functions. Naturally, throughout the implementation, the functional teams worked to design, test, and document their own functions with only minimal

cross-functional design and testing. We performed end-to-end integrated testing during two of the four major product testing cycles, with the project team members executing the tests. These team members were very knowledgeable about the software but were not always a part of the larger business design considerations.

Near the end of the program a decision was made to forgo user acceptance testing (UAT) and instead perform a business simulation in a few key pilot areas. The business simulation was led again by the project teams, but this time the end user was involved. End user documentation and training were created separately by each functional team.

A look back to the project revealed some implementation flaws that led to design issues and inefficiencies that could have been avoided. Some of the lessons learned from the project include:

- Spend more time designing and testing the bridges between the silos than designing the silos. Design teams should be composed of business, technical, and testing team members.

  Our story: Only a couple of weeks before go-live we found out that one functional team was creating purchase orders of a certain type, only to realize that the Accounts Payable team did not support that type of purchase order. Had we gone live with that design flaw, any purchase orders of that type issued to a vendor could not have been paid. This issue could have been avoided had we spent more time collaborating on the areas of integration between the functions.
- Train the end users using the end user documentation and training programs, and allow them to perform UAT with the knowledge provided. The project team should be involved only to help document the test results (including system, training, and documentation defects). This allows the users to test not only the system design, but also the end user documentation and training programs.

  Our story: We provided some documentation for the business simulation, but the training was performed by the project team members who knew and understood the design in detail. Not all end user documentation was available. The business simulation was performed by members of the project team with the end users. Although not realized at the time, the project team was performing work-arounds to the design flaws to allow the tests to pass. Had the end users performed all the tests themselves without assistance from project team members who had been working with the system for months, the design flaws would have been found much earlier. As well, since the project team members were readily available to answer questions during the business simulation, many documentation and training defects were not found.

As you can see from Susan's story, even the highest level of diligence on the part of the delivery teams might not be enough if you don't involve all stakeholder representatives in the testing activities.

ADVANTAGES OF REACHING OUT BEYOND THE DELIVERY TEAM

We started the chapter by talking about impacts and understanding the goals of the customer. When we adapt our processes to deliver to their needs, we not only meet, exceed, or even delight our customers; we usually end up with positive team dynamics. Join us in reading the final part of Geoff Meyer's story about Dell's journey.

Results—Dell, Part 5

The Enterprise Solutions Group is ultimately responsible for delivering value through our products, and our product deliveries are oftentimes tied to partner milestones.

The adoption of agile has provided Dell ESG with an increased level of schedule predictability. Within Engineering, the day-to-day work environment for programmers and testers has been greatly enhanced. Scrum teams are highly engaged, collaborative, and friendly and receive regular satisfaction from completing working features in a regular cadence. Development team members welcome and respect the skill sets brought by the test members of the Scrum team, and Test is now a full partner in the entire software development process. The inclusion of test engineers in the early requirements process, combined with their natural inquisitiveness about how features may work or may fail, has improved the usability and functionality of our products.

Within the Test organization, agile has led us to an advanced skill set profile for our testers. The necessity of test automation in support of agile has led us to acquire and/or develop software engineering skills. The automation skill sets, in turn, have led to cycle time reductions in test execution runs and increased test matrix coverage that would have not been conceivable prior to agile and automation initiatives. All in all, it's been a very productive journey for both the business and individuals.

Geoff's organization added different roles and specialist groups, such as test architects and an extended sprint test team, to enable the large and complicated organization to take advantage of agile principles and practices. They focused on solving one problem at a time and experimented with a variety of approaches to integrate coding and testing. It sounds as if the newly cross-functional team members enjoy collaborating, and each team member contributes his or her specialized expertise. Enjoying your work while delighting your customers is a great outcome of practicing continuous improvement.

No matter what the size of the organization or how many products it supports, agile principles apply. Teams in large enterprises are subject to a more extreme level of the difficulties we find with most teams practicing agile testing. For example, it's much harder to keep an eye on the big picture and identify ripple effects across a large system made up of multiple software products than with one application. It can be harder to communicate effectively with stakeholders since more of them are involved. There are different ways of tackling large-scale testing problems, but agile principles and values help guide you to try useful experiments, get quick feedback, and make good choices. The story of how Spotify scaled agile development is a good source of ideas on how to use agile principles without getting too rigorous (Kniberg and Ivarsson, 2012).

Summary

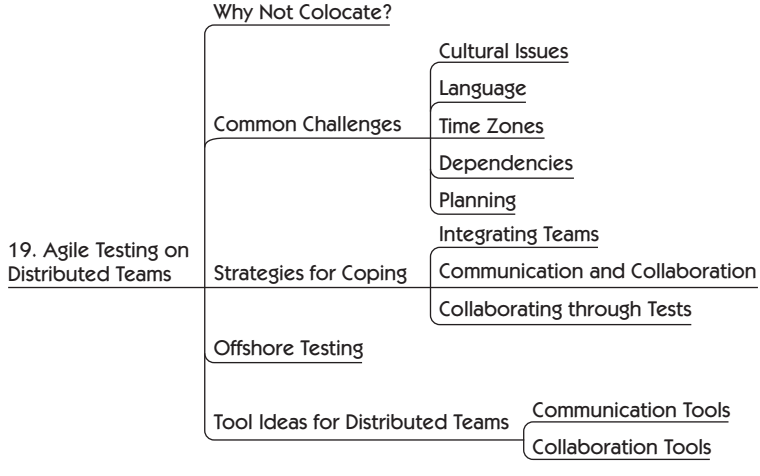
Enterprise environments magnify testing challenges for agile teams as they do for teams using any development process. You need more discipline, and more experiments to try, but still always starting with the simplest possible approach.

- Learn to work within the organizational controls, but try to find new solutions by offering suggestions. Understand the history and organizational culture so you can offer alternative solutions.
- Specification techniques such as Planguage may be useful to define goals and use consistent language when describing quality attributes.
- When large enterprises implement agile, it helps to have a champion at the executive level to help support teams in the transition.

- Fast feedback is critical in enterprise solutions when many teams and applications need to work together. Understand the big picture and the dependencies between teams to determine the best way to give and receive feedback.
- Managing dependencies is challenging in large enterprises. Make the dependencies visible to encourage thinking about them during planning.
- Consider adding additional roles such as test architects or test managers to help coordinate activities among many teams.
- Consider a system test team to take advantage of economies of scale in test environments.
- Use consistent tooling; CI and version control help multiple teams working on the same or related products coordinate.
- Reach beyond the single delivery team and the tests that they can do. Include other members of the organization, customers, and end users to make the solution complete.

Chapter 19

Agile Testing on Distributed Teams



Almost every book that we have read on scaling large or distributed teams recommends that feature teams exist in one location if possible. This is seen as a prerequisite to the collaboration that is so necessary in agile. However, we recognize that it is difficult, if not impossible, for some organizations to change their existing distributed structure. There may even be advantages to their distributed nature that they'd rather not give up.

We make a distinction between dispersed teams, distributed teams, and offshore testing, but we will use the term *distributed teams* to mean all three, unless we are specifically talking about dispersed or offshore.

- **Dispersed teams** may have several team members working remotely from home, although there may also be some colocated team members. They work together as one team (see Figure 19-1).

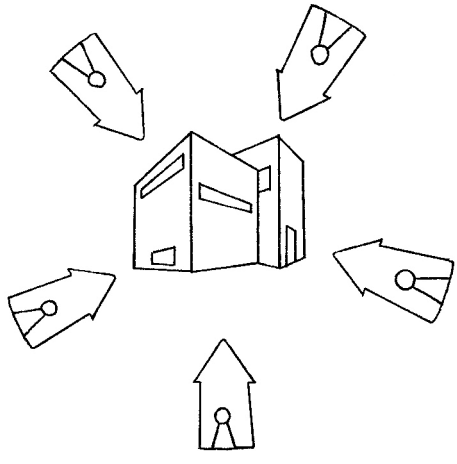


Figure 19-1 Dispersed teams representation

- **Distributed teams** are those that have sub-teams or full teams colocated in multiple locations, possibly in different time zones or different countries, all working on the same product (see Figure 19-2).

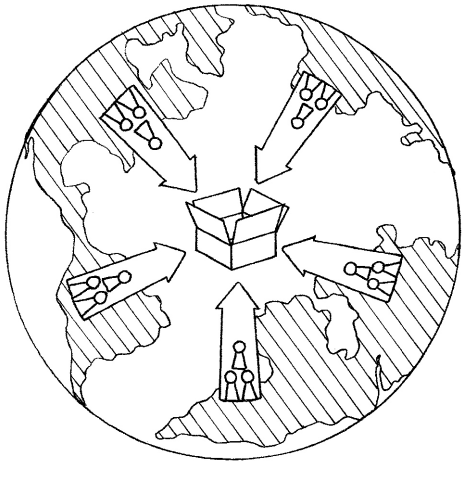


Figure 19-2 Distributed teams representation

- **Offshore testing** is when an organization chooses to subcontract its product testing to a vendor in another location, usually in another country where labor costs are lower. Near-shoring is a variation of this, where testing is outsourced to countries in a closer location. For example, a U.S. company might near-shore with a company in South America or Mexico. Near-shoring usually means there is no, or little, time zone difference.

There are other variations on these categories. A company might keep testing at its main location but offshore some development work. The recommendations in this chapter still apply.

We consider ourselves a dispersed team. Both of us work remotely but work as one team with no home base. While writing this book, we used many of the tools we suggest to you in this chapter. For example, we used Google Docs and Dropbox to share information to which we both needed real-time access, such as our release plan. We used MindMeister for our mind-mapping sessions, sometimes collaborating in real time on an idea, and other times each of us making updates to share with the other later. When we needed to connect and have real-time conversations, we used Skype or Google Hangouts. We tested our ideas and tried to get fast feedback by sharing chapters with reviewers on Google Docs, and we used a Google Group for reviewers to share ideas or to collaborate on clarifying something that might be confusing. It may not have been the ideal situation, but we made it work (Gregory, 2014).

There are many books and resources to help you make your distributed team successful. The difficulties aren't confined to testing. However, it's much more difficult to collaborate on testing activities and to share collective responsibility for quality on distributed teams. In this chapter we'll look at how testing can succeed on distributed teams.

Why Not Colocate?

There are several reasons why organizations have distributed teams. Often it is due to mergers or acquisitions as the company has grown. One of the main reasons that teams have remote team members is because they cannot find qualified candidates in their geographical area. For example, it is becoming more and more difficult to find good testers

in some areas of the world. Good people help software projects succeed. Some teams have changed their norms to accept people who work from elsewhere. In these instances, the disadvantages of non-colocation are offset by the value that remote employees bring to the team.

There are advantages to distributed teams, but this next section will cover testing-related challenges that distributed teams face, and we'll offer some suggestions to help overcome those challenges.

COMMON CHALLENGES

It doesn't matter if you have globally distributed teams or individual remote team members; learning to work together from multiple physical locations is difficult. For new agile teams where testers are learning to collaborate with programmers, customers, and other team members, physical distance makes the transition more difficult.

Cultural Issues

Teams distributed around the world need to deal with cultural issues. For example, people from one country (let's call it Reservandia) may be quiet and reserved, while in another country (we'll call it Assertivana) the people speak their mind.

Take a moment to think about these two perspectives. We expect these two groups to work as one team. The programmers are in Assertivana, and the testers are in Reservandia. The programmers start to talk about how they think the software should be implemented. The testers sit back and listen; they may feel that they don't even have a chance to say a word. This may not always be the case, but we have heard this same story over and over, from both sides. The programmers from Assertivana complain that the testers never speak up, while the testers say they never have the chance to offer suggestions.

These are troublesome differences, but with some coaching on both sides, they can be overcome. First, both sides must feel safe to share their concerns or suggestions, especially Reservandia. Assertivana's programmers can learn to listen first and then give their opinions. Reservandia's

testers can be prepared and perhaps introduce their ideas through email to start with, and then be prepared to speak on the subject.

Companies that choose to offshore certain development activities such as testing should be aware of the impact that decision can have on team culture, both in the main location and for the offshore team. As Jan Petter Hagberg pointed out to us (Hagberg, 2013), the team members "back home" probably didn't choose to work with an offshore team. The process of offshoring may have been traumatic for the team members still in the original location. Make sure they get help through this transition, which can take a long time.

Cultural differences can cause a comment intended as a joke by one person to be insulting to another. Slang also can get in the way of smooth communication. Take the time to learn about the team members in other locations and their culture. Experiment with ways to improve communication.

A word of caution: too often we see teams use the term *cultural issues* as an excuse not to look for a solution.

Language

Teams that span countries face language problems. English is the most commonly used language, but it is not everyone's first language, and when spoken, it is often with an accent. Even with English as a first language, the speaker's accent and speed of talking affect how others hear. Slang and colloquialisms within the same language also can create misunderstandings. Creating a common language when talking about stories and features is even more important for distributed teams.

Janet's Story

I spend a lot of time in other countries training and consulting. In one training session in Denmark, one of the participants came to me and complimented me on speaking slowly and using common words, but he said that he probably missed 5% to 10% of what I said. I was surprised because he spoke excellent English. We discussed it further, and it made me realize the impact of communication within teams.

Let's consider three teams: one in the United States, one in China, and one in France. When the folks in France speak with a French accent, and maybe

struggle to find the right words, the people in the United States probably lose the typical 5% to 10% in translation. What do you think happens when the team in China is trying to understand? How much do we really come to a common understanding when we are all losing a little bit, or perhaps a lot?

Awareness is half the battle, but we can help ourselves and our teams make it better. Try things like repeating an idea, or paraphrasing, to make sure you understood, or having a spokesperson repeat the sentence or concept. People need to feel that it is safe to say they didn't understand and ask someone to repeat a comment. Give people time to mentally translate what's been said before moving on. Perhaps summarize decisions in writing on a team wiki so team members can read and have time to internalize. Think before you ask a question. "Let's take a poll to get everyone's opinion" instead of, "Anybody have any objections?" will help determine if everyone understood the question. Of course, it still might not work the way you would like, but you have a better chance. Lars Sjödahl gave us that little piece of valuable advice during his lightning talk "The Damage Done by Acts of Silence" at Agile Testing Days 2013.

In the section "Strategies for Coping" later in this chapter, we'll give some testing ideas that may help you win this battle.

Time Zones

One of the main problems that distributed teams face is time zones. When one sub-team, or the entire testing group, is 12 hours away, it probably means there is little or no verbal communication, unless someone works late into the evening. Some organizations arrange for one team to work at night to match hours with another. Others have one team member join a daily standup remotely late in the evening or early morning to try to keep some level of verbal communication.

When face-to-face communication isn't an option, teams must find a way to describe their needs so that one team does not sit idle, perhaps because they didn't know what to do next. Another example of problems faced by distributed teams is the lack of environments for testing.

Support needs to be available for all teams, all the time, if the teams use a common code base and continuous integration (CI) environment.

Janet's Story

In one organization we had teams in two different countries and practiced continuous integration so that testers living in Europe could come to work in the morning and pick up stuff to test. One of the problems we discovered was that they didn't always have people available to answer questions, so the actual feedback loop was lengthened. One of the work-arounds we put in place was nightly emails—what we had worked on, what was outstanding, what issues existed. We made them as complete as possible.

There was a seven-hour time difference, so when it was 4:00 p.m. their time, it was 9:00 a.m. our time. We tried to have people come in early (7:30 a.m.) so we had at least a couple of hours of overlap if we needed to talk in person. These days, I would probably have a Skype call every morning.

Dependencies

Be aware of the dependencies between groups. If you are waiting for a deliverable from people in a different time zone, and they don't have it ready, you may have to wait at least another 24 hours. Team members may start other tasks while they are waiting, which results in task switching among multiple commitments. Discuss potential dependencies in planning meetings, and find ways to make them visible or eliminate them. Online tracking tools can be used to show whether testers are waiting for stories to be delivered or are getting them all at once, or if too many stories are waiting for someone to accept them.

Planning

Planning sessions are especially difficult when your teams are not all in the same location. It is helpful to gather the whole team physically in one place for the release kickoff or project inception meetings. Teams that have done this have seen significant benefits in how the team works when team members are back in their respective locations. In Chapter 9, "Are We Building the Right Thing?," and Chapter 11, "Getting Examples," we talked about how testers can help by testing business value while it's still in the idea stage. Those benefits apply to distributed teams but are much more difficult to realize if the whole team is not in one place.

Experiences Working on Distributed Teams

Huib Schoots, *who works at Improve Quality Services in the Netherlands, shares what he's learned working on distributed teams.*

"Coming together is a beginning. Keeping together is progress. Working together is success." —Henry Ford

I've worked on many international projects in my career. And without exception, projects always went more smoothly when I was on-site. Collaboration improved, communication was much better, and work got finished faster. A project I did in Jakarta had the same pattern; at the start I was there for only a week every month, and later I was there full-time. The collaboration improved dramatically! Why? Because we got to know each other better and we took time to overcome our cultural differences.

When not in the same physical location, people can hide more easily and tend to isolate themselves. Communication gets more difficult and less rich. The things everyone is working on become less transparent, and to overcome this issue we use our tools in combination with more comprehensive documentation. In my experience, teams rely too much on documentation and tooling. These can actually make it harder to keep everything up-to-date, and information isn't shared as easily.

People need to run into each other to be really successful. Organizations need to make mixing as simple and obvious as possible. Get people together on a regular basis; give them time to build trust and get to know each other. Effective communication is in the details, and to recognize these you have to know your team, so spend as much time together as possible.

In my experience, the most important thing is to acknowledge that communication is the biggest risk in the project. When everybody is aware of this risk, teams have the best chance of overcoming the difficulties they will have. Teams will put in deliberate effort to make sure that communication is effective; they need to be creative to make the best of what they have.

We used Skype and Google Hangouts a lot, but there are many tools available to facilitate communication online. We used video to see each other, and this helped us to get to know people better. You at least have the feeling you know who you are talking to. We also prepared for our meetings; the team leads from each location got together (online) before the actual meeting to discuss the highlights.

By doing this we always got a good agenda for each meeting, and we invited only the people who really needed to be there. Another thing we learned from videoconferences is that those meetings take longer than normal face-to-face meetings. As well, in-depth discussion with big groups is very difficult; therefore, we avoided it. An advantage of using videoconferencing is that you can record the meeting and play it back to people who weren't able to attend.

We experimented with online agile boards, but in our situation they didn't really work out. They were handy in the communication with the other location, but they did not work in my team on location. We didn't have an overview of the stuff we were working on, so we used a whiteboard as our agile dashboard and updated the online tool daily before the meeting with the other teams.

To create team spirit we tried webcams at each location so we could see what was happening on the other side. This has an extra advantage, since you can see who is available to talk to. It worked out quite well.

Testing Considerations

Testing had to cope with communication difficulties as well, and asking questions, sharing information, and pairing became more difficult. It is harder to involve programmers in a different location to show them your findings or ask questions. Technology helps, but we still had to write down findings and use the bug tracker more often than we would normally. Pair testing became distributed, and using screen-sharing tooling via the web partially solved this problem. After we found out that pair testing was working quite well using tooling, communication improved and became more frequent.

Making It Work

To build good relations I encourage distributed teams to get together as much as possible. It helps to get to know each other, to get acquainted with culture and work ethics, but also to optimize communication. It is especially important to use face-to-face communication for retrospectives and in-depth discussions.

Distributed teams can work, but be aware of the consequences. Try several ways of communication and discuss and evaluate them often. Adapt them when they are not working. Working together is about relationships, communication, and people. When people are far away from each other but still have to work together, there are many ways, including various tools, to make that possible. Yet, no matter how well the tools are working and no matter how passionate and talented your colleagues are, distributed teams are always suboptimal.

Huib shares some good strategies to enable team members to collaborate for testing and has provided some links that we included in the bibliography for Part VII. Let's look at some more ways to make agile testing possible in distributed teams.

STRATEGIES FOR COPING

Ideally, your distributed team members can regularly meet in one location to discuss the challenges that geographical distance presents. If this isn't possible, you can still find good ways to communicate and build relationships. There are plenty of tools available to enable remote pairing and group collaboration. See the "User Research" story of Disney's user experience designers collaborating remotely in Chapter 13, "Other Types of Testing."

Integrating Teams

We, along with other practitioners, recommend that team members in different locations travel as often as possible to maximize "face time" and shared experiences. This is the best way to build the trust needed to work together productively. We recommend trying to cluster self-sufficient teams around features if you can, to reduce the dependency problem and help the team feel more cohesive. Make sure all team members have access to business stakeholders so they can get questions answered quickly. Having experienced coaches, team leads, or subject matter experts in each location helps the cohesiveness of the teams.

Team members in the various geographical locations should agree on the same definition of "done." We have seen teams in one location think that Story Done meant coded and tested completely, while the other team, on which they depended, thought it referred only to completed code, ready for integration testing. This caused issues and a great deal of antagonistic feelings between teams.

Extra time and discipline are needed to help everyone understand responsibilities and accountabilities. You may spend extra time identifying how people interact, how decisions are made, and how the work is split between teams. Try keeping the tasks and stories small enough that they can be completed in a timely manner for the other teams to work with.

Lack of trust is one of the greatest obstacles that distributed teams face. They can't get together for team-building exercises, drinks, or a meal. They have no easy way to establish the relationships necessary for building trust. In lieu of face-to-face contact, take the time to learn about the other remote teams, their cultures, and their work area. Find some way to have social interaction online. Create a wiki page where people can post pictures and share some personal information. Set up a space for personal blogs so team members can tell personal or professional stories. Have a team chat channel devoted to social exchanges and sharing jokes. Play online video games together.

Lisa's Story

My current team is separated by only two time zones. Even so, we need a lot of discipline to maintain good communication. Programmers pair for all production code, and we also often pair between roles, such as a tester with a designer, or a designer with a programmer. Every team member diligently keeps an eye on the team chat channel in case someone can answer a question or just wants to share a funny picture. Sometimes on Friday afternoons, team members play a collaborative video game or watch fun video clips together.

But what's probably most important is that every couple of months, the entire team gathers in one place for a week. During this week we have social activities together and an in-person team retrospective. In between those times, individuals travel to other locations to work. The travel is expensive, but the increased level of trust and ability to communicate enables us do better work, more efficiently. With few testers on the team compared to the number of programmers, creating an atmosphere of comfort and trust is especially helpful.

Communication and Collaboration

When teams span the globe, they need to find ways of communicating that span language, time zones, culture, and communication styles. The whole team needs to be aware of all tasks that are being worked on, including testing activities. Collaboration tools must work equally for all team members. In colocated teams, team members can have spur-of-the-moment conversations. Distributed teams do not have this luxury. They need to create structured conversations and find a way to share information with all team members in all locations.

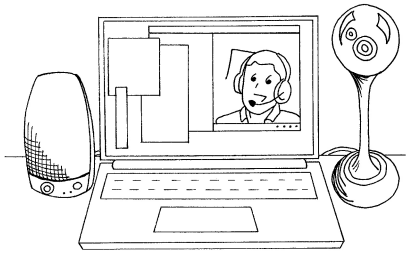


Figure 19-3 Take advantage of technologies—create a virtual you.

When most of the team is in one location but some team members are working from another location, perhaps from home, it is critical that those team members have some face time with the rest of the team. Lisa has been on dispersed teams, sometimes as one of the remote team members, and has experienced the positive effects of face-to-face communication—even if it is virtual. Lisa felt part of the team when she was included in the conversations. One of her teams created a "Virtual Lisa," using a telepresence device, much like the one in Figure 19-3. They rolled her into iteration-planning meetings, daily standups, and to whoever was her pair for the day. Virtual Lisa consisted of a laptop on a rolling cart, with a good-quality microphone that could pick up even casual conversations in the room, and a webcam that could be controlled remotely. This made Lisa feel truly part of the team in every way because she could participate in conversations and not worry that she missed something.

Agreeing on a telecommuting policy helps promote the kind of culture and discipline needed for teams with remote members to work effectively. Chris McMahon outlined an example policy in his "Telecommuting Policy" blog post (McMahon, 2009). Transparency and visibility may be even more important on distributed teams than on colocated teams. Think about what information you want to share and what is the best way to share it. How can you convey essential information at a glance? Can you make a simple test matrix to show testing progress for a release and put it on the landing page of the project wiki? Lisa's previous team took a picture of their task board every day and put it on the wiki so the remote team members could see the current status of the physical story board.

In his "Visualizing Quality" Agile Testing Days 2013 Keynote (Evans, 2013), David Evans recommended showing each team member's picture (Figure 19-4) along with their stories and tasks on the kanban system or

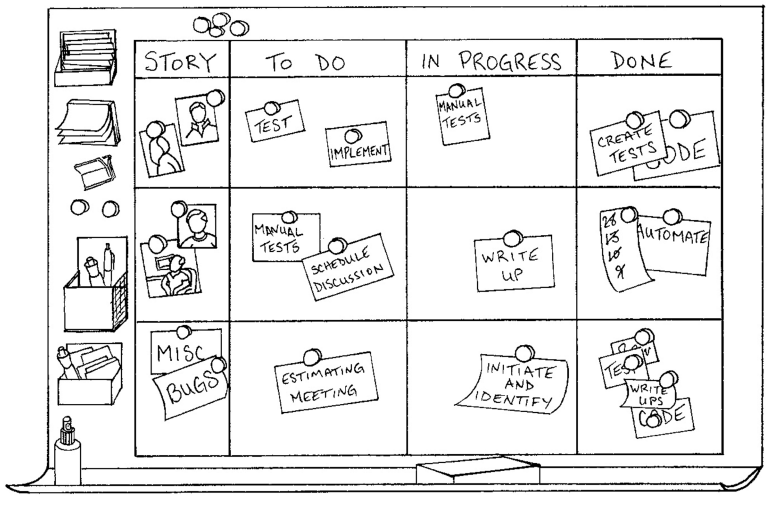


Figure 19-4 Get to know your distributed team members.

story board. This is particularly helpful for distributed teams. It helps everyone visualize who is anchoring each story or task, and it also shows when someone doesn't currently have work to do. We've met teams that include a photo or avatar of the person anchoring each story on their online project-tracking tool. They reported that it helped with communication and helped prevent stories from falling into black holes. Lisa's previous team did this with their physical task board and had some fun with the pictures. Since photos of the task board were posted to the wiki every day, remote team members got to join in the fun.

Collaborating through Tests

Examples and tests are powerful ways to communicate, but teams often fail to take advantage of them. Tests can be written in a domain-specific language (DSL) that crosses language barriers. Ideally, team members can collaborate at the same time, but that isn't always possible. If testers are in one country and programmers in another, perhaps the testers can write tests, which the programmers can review prior to discussions about the story. By starting early when there's a big time zone difference, team members can be prepared and use the tests to spark discussions. Product owners should be part of these discussions, too, vetting the examples and the tests and continuing the conversations until the whole team has a shared understanding of the story.

If the time zone difference allows, try remote pairing to keep the feedback cycle short. Janet listened to an experience report on remote pairing at Agile 2013 by Johannes Brodwell (Brodwell, 2013) which described testing that was offshored to Sri Lanka, while the rest of the delivery team and the customers were in Sweden and Norway. Time zones were not an issue, so team members were able to connect remotely and collaborate through pairing. They used tools such as Skype to be face-to-face as much as possible. Screen- and file-sharing tools such as GoToMeeting and Dropbox eased collaboration. To keep paired programming moving more quickly, they used a Ping-Pong approach to test-driven development (TDD): one programmer writes a test, and the other programmer writes the code; then they switch roles. This approach could work for testing activities such as specifying tests or debugging test failures. One of the biggest benefits this company found was that the pairs got to know each other better. They learned each other's idioms and preferences, and their mutual respect grew. Their process provided real-time training for new team members. Domain knowledge was passed on quickly, as was business understanding.

OFFSHORE TESTING

Some organizations believe that offshore testing is cheaper, because the perceived cost per person-unit of testing is lower. However, the overhead in extra layers of communication and the miscommunication that happens between people whose first language is not the same might outweigh the cost savings in labor. Often offshore testing means "throwing code over the wall" for testing and is not really about getting fast feedback.

Janet's Story

I was recently talking to a globally distributed organization (Company A) that was trying to move to agile. The company's multiple applications were very tightly coupled, yet owned by multiple vendors as well as the company's own internal agile development teams that supported the front end. One of the vendors was using a flow-based system to develop its application, but because it was delivered to many different clients, the timing did not always match Company A's needs. The other vendors were using traditional phased and gated methods for development. An internal test team handled the integration testing, but user acceptance testing (UAT)

was outsourced to another vendor whose testers worked both on-site and offshore to keep costs lower for the client.

They faced many issues, including how to get features delivered when multiple applications and vendors were involved, and they had to rely on requirements and specification documents. Company A wanted to take advantage of "following the sun"—that is, having work constantly in progress by a team in at least one time zone. Extra effort was needed to make sure that the handover from one group to the next was seamless. For example, they created a position in the North America office to work with the offshore testing team to act as a liaison between the teams. They also had a person on call so that there was support for the offshore team if the test environment became unavailable.

I tell this story to show the difficulties some teams face when an organization does not understand the cultural and structural changes necessary to enable the teams to take advantage of agile. This organization has a difficult time ahead to align all these groups and vendors to deliver in small chunks but is determined to make it work.

One way to make offshore testing work with an agile approach might be to involve a system integration team from the beginning and give the full workflows to the teams so that there is consistency in delivery. Perhaps specific flows could be developed in priority order, using the "steel threads" approach described in *Agile Testing* (p. 144), working on thin end-to-end slices incrementally. This allows the flows to be tested earlier to reduce the risk. This may prevent, or at least mitigate, the problem where one team delivers its part of the system but breaks all other applications associated with the capability.

Developing multiple applications concurrently increases the likelihood of integration problems. In Chapter 18, "Agile Testing in the Enterprise," we mentioned the idea of a system integration team. Complex distributed organizations may warrant having such a team that continually tests the overall system, including all the enterprise applications. In Janet's example, perhaps a virtual team made up of members from both the integration and the UAT teams would ensure that everyone understands what is being delivered on an iterative basis.

Many companies that offer testing as a service have started to offer agile testing. If these organizations find ways to provide fast feedback, they